

FDTT 704: Principal Component Analysis

Dr. Mutua Kilai, PhD

September-December 2026

Department of Pure and Applied Sciences

Introduction

- Principal component analysis (PCA) is a method of data reduction or compression that is used to turn a conceivably large dataset of (potentially) correlated variables (or questionnaire items) into a smaller number of uncorrelated variables known as principal components.
- The idea is that each of the n observations lives in p -dimensional space, but not all of these dimensions are equally interesting.
- In PCA we look for a smaller number of dimensions that are as interesting as possible, where the concept of interesting is measured by the amount that the observations vary along each dimension.

Cont'd

- Each of the new dimensions found in PCA is a linear combination of the original p features.
- The hope is to use a small subset of these linear feature combinations in further analysis while retaining most of the information present in the original data.

Purpose of PCA in Fashion Design

Text

In fashion and textile research, datasets often contain many interlinked measurements—such as multiple body dimensions in anthropometric sizing, various fabric mechanical properties and washing effects, or high-dimensional pixel data from clothing images. The purpose of PCA is to transform these large sets of correlated variables into a smaller, manageable set of uncorrelated new variables called principal components. This helps researchers simplify complex data, eliminate redundancy, and easily visualize similarities or groupings among garments, textiles, or consumer preferences without losing vital information.

Appropriate Situations for PCA

Body Measurement and Sizing Studies

- When analyzing extensive 3D body scan data containing dozens of highly correlated human body dimensions to define standardized size charts or representative body shapes for clothing production.

Textile and Fabric Performance Evaluation

- When evaluating multiple overlapping physical and mechanical properties of fabrics (such as breaking strength, tear strength, drape, and shrinkage after washing) to discover the core factors driving fabric quality.

High-Dimensional Image and Visual Analysis

- When processing large collections of clothing images or visual catalog data where pixel features or shape descriptors create too many dimensions for standard computer vision or machine learning models.

Consumer Preference and Market Segmentation

- When surveying consumer attitudes with dozens of heavily correlated rating scales regarding style, price, comfort, and brand loyalty, and you need to bundle them into core underlying lifestyle or purchasing drivers

Finding Principal Components

- The first principal component of a set of features $X_1, X_2, \dots, X_p$ is a linear combination of the features

$$Z_1 = \phi_{11}X_1 + \phi_{21}X_2 + \dots + \phi_{p1}X_p$$

that has the largest variance.

- Here $\phi_1 = (\phi_{11}, \phi_{21}, \dots, \phi_{p1})$ is the loading vector for the first principal component
- The ϕ are normalized so that $\sum_{j=1}^p \phi_{j1}^2 = 1$.

Cont'd

- After the first principal component Z_1 has been determined we can find the second principal component Z_2 .
- The second principal component is the linear combination of that has maximal variance out of all linear combinations that are uncorrelated with

where we again define as the loading vector for the second principal component

- This process proceeds until all p principal components are computed.

Example in R

```
1 library(dplyr) # basic data manipulation and plotting
2 library(ggplot2) # data visualization
3 library(h2o)    # performing dimension reduction
```

- To illustrate dimension reduction techniques, we'll use the my_basket data set.
- This data set identifies items and quantities purchased for 2,000 transactions from a grocery store.
- The objective is to identify common groupings of items purchased together.

Cont'd

```
1 url <- "https://koalaverse.github.io/homlr/data/my_basket.csv"
2 my_basket <- readr::read_csv(url)
3 dim(my_basket)
```

```
[1] 2000  42
```


Cont'd

- To perform dimension reduction techniques in R, generally, the data should be prepared as follows:
 1. Data are in tidy format per Wickham and others (2014);
 2. Any missing values in the data must be removed or imputed;
 3. Typically, the data must all be numeric values (e.g., one-hot, label, ordinal encoding categorical features);
 4. Numeric data should be standardized (e.g., centered and scaled) to make features comparable.

Performing PCA in R

- There are several built-in and external packages to perform PCA in R.
- We recommend to use h2o as it provides consistency across the dimension reduction methods we'll discuss later and it also automates much of the data preparation steps previously discussed.

```
1 h2o.no_progress() # turn off progress bars for brevity
2 h2o.init(max_mem_size = "5g") # connect to H2O instance
```

H2O is not running yet, starting it now...

Note: In case of errors look at the following log files:

```
C:\Users\MUTUAS~1\AppData\Local\Temp\RtmpGEaKSF\file8245ecaa62/h2o_Mutua_Sam_started_from_r.out
C:\Users\MUTUAS~1\AppData\Local\Temp\RtmpGEaKSF\file82410823a20/h2o_Mutua_Sam_started_from_r.err
```

Starting H2O JVM and connecting: Connection successful!

R is connected to the H2O cluster:

```
H2O cluster uptime:      8 seconds 334 milliseconds
H2O cluster timezone:    Africa/Nairobi
H2O data parsing timezone: UTC
```



```
1 # convert data to h2o object
2 my_basket.h2o <- as.h2o(my_basket)
3
4 # run PCA
5 my_pca <- h2o.prcomp(
6   training_frame = my_basket.h2o,
7   pca_method = "GramSVD",
8   k = ncol(my_basket.h2o),
9   transform = "STANDARDIZE",
10  impute_missing = TRUE,
11  max_runtime_secs = 1000
12 )
```

Cont'd

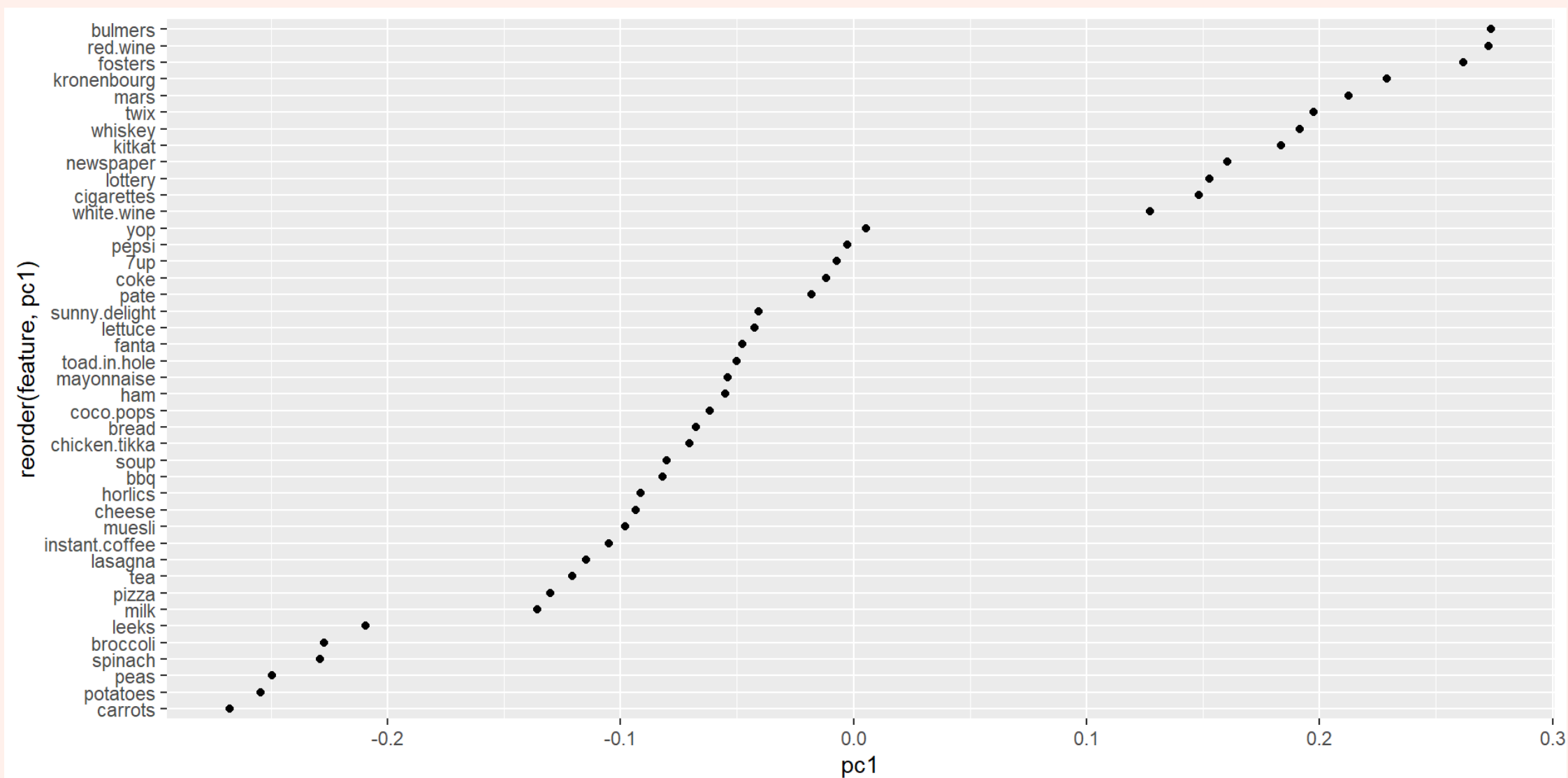
```
1 # convert data to h2o object
2 my_basket.h2o <- as.h2o(my_basket)
3
4 # run PCA
5 my_pca <- h2o.prcomp(
6   training_frame = my_basket.h2o,
7   pca_method = "GramSVD",
8   k = ncol(my_basket.h2o),
9   transform = "STANDARDIZE",
10  impute_missing = TRUE,
11  max_runtime_secs = 1000
12 )
```

Cont'd

```

1 my_pca$model$eigenvectors %>%
2   as.data.frame() %>%
3   mutate(feature = row.names(.)) %>%
4   ggplot(aes(pc1, reorder(feature, pc1))) +
5   geom_point()

```



Selecting the number of principal components

- So far we have computed PCs and gained a little understanding of what the results initially tell us.
- However, a primary goal in PCA is dimension reduction (in this case, feature reduction).
- In essence, we want to come out of PCA with fewer components than original features, and with the caveat that these components explain as much variation as possible about our data.
- But how do we decide how many PCs to keep?

Cont'd

- There are three common approaches in helping to make this decision:
 1. Eigenvalue criterion
 2. Proportion of variance explained criterion
 3. Scree plot criterion

Eigenvalue Criterion

- The sum of the eigenvalues is equal to the number of variables entered into the PCA; however, the eigenvalues will range from greater than one to near zero.
- An eigenvalue of 1 means that the principal component would explain about one variable's worth of the variability.
- The rationale for using the eigenvalue criterion is that each component should explain at least one variable's worth of the variability, and therefore, the eigenvalue criterion states that only components with eigenvalues greater than 1 should be retained.

Cont'd

```
1 # Compute eigenvalues
2 eigen <- my_pca@model$importance["Standard deviation", ] %>%
3   as.vector()
4
5 # Sum of all eigenvalues equals number of variables
6 sum(eigen)
7 ## [1] 42
8
9 # Find PCs where the sum of eigenvalues is greater than or equal to 1
10 which(eigen >= 1)
```

Proportion of variance explained criterion

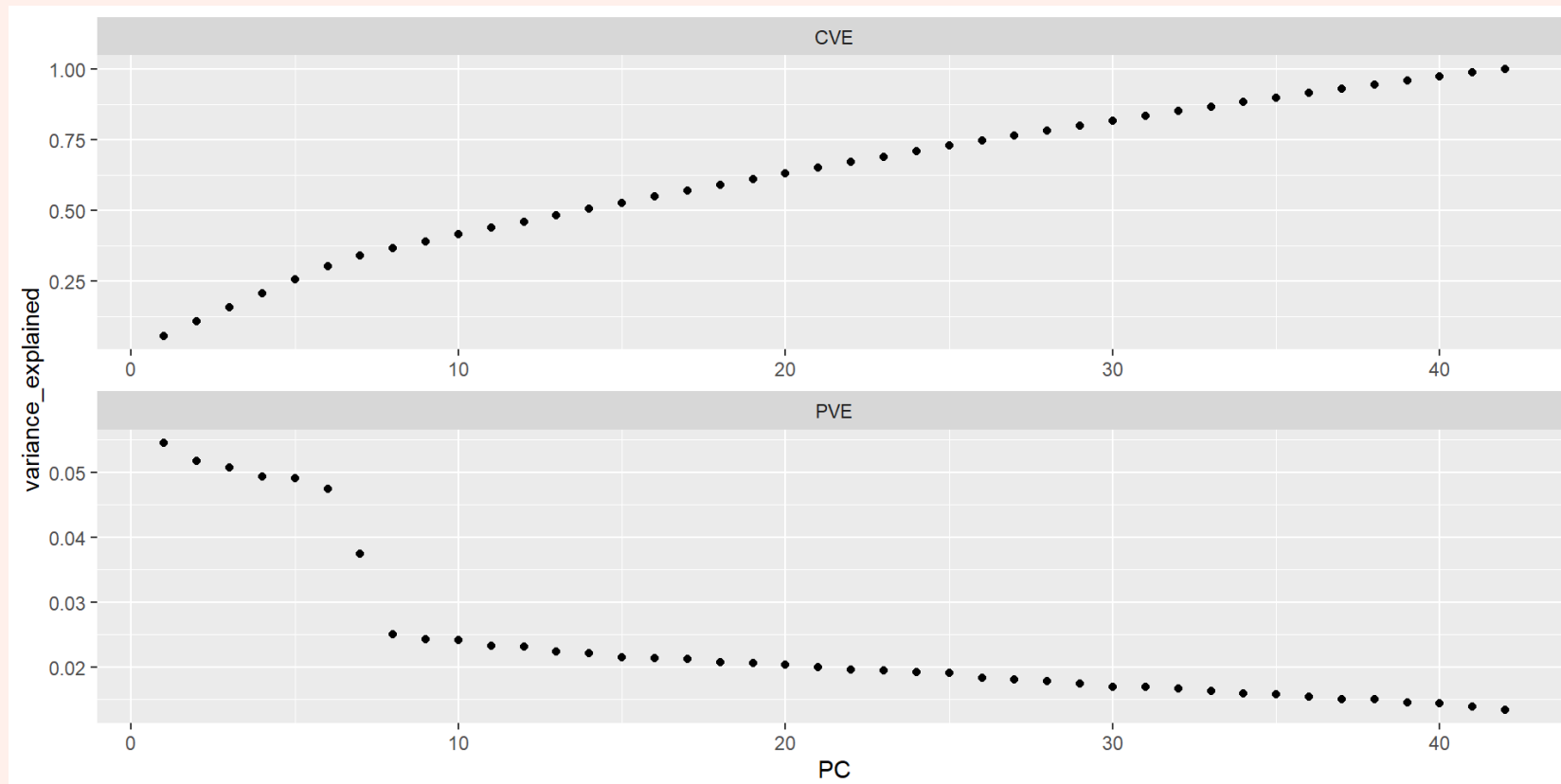
- The proportion of variance explained (PVE) identifies the optimal number of PCs to keep based on the total variability that we would like to account for. Mathematically, the PVE for the m-th PC is calculated as:

Cont'd

```

1 # Extract and plot PVE and CVE
2 data.frame(
3   PC = my_pca@model$importance %>% seq_along(),
4   PVE = my_pca@model$importance %>% .[2,] %>% unlist(),
5   CVE = my_pca@model$importance %>% .[3,] %>% unlist()
6 ) %>%
7   tidyr::gather(metric, variance_explained, -PC) %>%
8   ggplot(aes(PC, variance_explained)) +
9   geom_point() +
10  facet_wrap(~ metric, ncol = 1, scales = "free")

```



Scree Plot Criterion

- A scree plot shows the eigenvalues or PVE for each individual PC. Most scree plots look broadly similar in shape, starting high on the left, falling rather quickly, and then flattening out at some point.
- This is because the first component usually explains much of the variability, the next few components explain a moderate amount, and the latter components only explain a small fraction of the overall variability.
- The scree plot criterion looks for the “elbow” in the curve and selects all components just before the line flattens out.

Cont'd

```

1 data.frame(
2   PC = my_pca@model$importance %>% seq_along,
3   PVE = my_pca@model$importance %>% .[2,] %>% unlist()
4 ) %>%
5   ggplot(aes(PC, PVE, group = 1, label = PC)) +
6   geom_point() +
7   geom_line() +
8   geom_text(nudge_y = -.002)

```

